

Langages et traducteurs

Notions de bases

Atindehou Mêton-Mêton ^{δ}

^{δ} Université de Parakou
Institut Universitaire de Technologie

Novembre 2022

Plan

1. Langages formels
2. Grammaires formelles de Chomsky

Plan : 1 - Langages formels

- 1.1 Formalisation des notions de problèmes et d'algorithmes
- 1.2 Langages formels
- 1.3 Langages réguliers
- 1.4 Langages rékursifs, langages récursivement énumérables

L'idée de la compilation

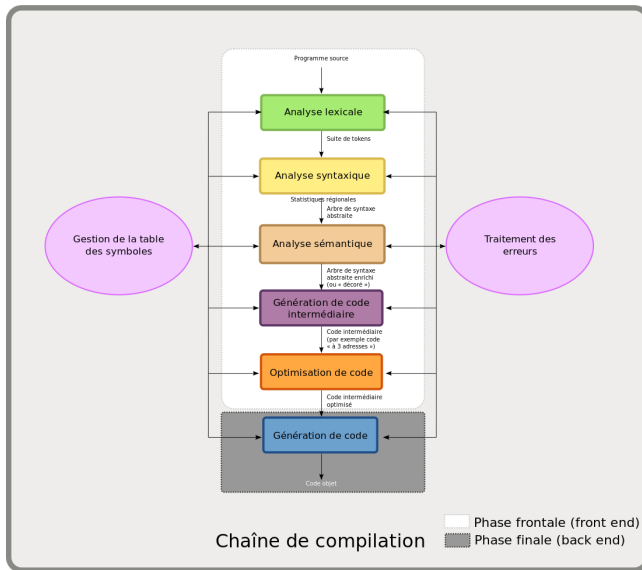


Figure: <https://fr.wikipedia.org/wiki/Compilateur>

Formalisation des notions de problèmes et d'algorithmes

A quoi ça sert de formaliser la notion de problème ?

Nous avons une question (problème) du monde réel et nous cherchons un **procédé mécanique et automatisable [algorithme automatisable]** dans “le monde informatique” pour déterminer les réponses à cette question.

Définition (Procédé mécanique et automatisable)

Un procédé mécanique et automatisable est un procédé qui s'écrit sous la forme d'un algorithme programmable sur un ordinateur, et de manière que le programme correspondant s'achève toujours en donnant “une” réponse à la question posée.

Remarque

- Le terme “procédé mécanique” est emprunté à Quine.
- Un “*procédé mécanique et automatisable*” est aussi désigné dans la littérature par “*méthode effective de calcul*” ou encore par “*algorithme automatisable*”.

Cheminement possible pour résoudre les problèmes

Proposition de Pierre Wolper ¹ :

- Déterminer tout d'abord si une question donnée fait partir de la classe des problèmes solubles à partir d'un algorithme automatisable.
- On a besoin de *définir précisément* (ou formaliser) :
 - ▶ La notion de problème,
 - ▶ La notion d'algorithme automatisable.
- **Pourquoi** : éviter de perdre du temps à chercher un algorithme automatisable pour une question qui n'en-a-pas !

¹Introduction à la calculabilité, Dunod, 3ieme édition.

Notion de méthode effective de calcul

Source : https://fr.wikipedia.org/wiki/Th%C3%A8se_de_Church

Définition (Méthode effective de calcul - wikipedia)

“La notion intuitive de méthode effective de calcul correspond aux caractéristiques suivantes :

- 1. l'algorithme consiste en un ensemble fini d'instructions simples et précises qui sont décrites avec un nombre limité de symboles ;*
- 2. l'algorithme doit toujours produire le résultat en un nombre fini d'étapes ;*
- 3. l'algorithme peut en principe être suivi par un humain avec seulement du papier et un crayon ;*
- 4. l'exécution de l'algorithme ne requiert pas d'intelligence de l'humain sauf celle qui est nécessaire pour comprendre et exécuter les instructions.”*

Notion d'algorithme automatisable

Définition (Algorithme, algorithme automatisable)

- Un **algorithme** est une procédure de résolution d'un problème.
- Un algorithme est **automatisable** lorsqu'il est systématique, mécanisable et implémentable sur un automate appelé ordinateur.

Dans la suite, un algorithme automatisable sera plus simplement appelé algorithme.

Notions de problème

Définition (Notions de problème)

- Un **problème** est une question posée sur un ensemble de valeurs données.
- Une **instance d'un problème** est une question posée pour une valeur particulière.
- Les instances d'un problème doivent être représentées pour être des entrées des algorithmes automatisables : **ce seront plus simplement des chaînes de caractères.**
- Une **fonction d'encodage** construit la représentation d'une instance de problème sous forme de chaîne de caractères.

Remarque

Le format de représentation des données par des chaînes de caractères généralise les formats de type de données usuels. Car, il est possible de passer d'un format à un autre en utilisant des méthodes appelées pour cela des **passeurs**.

Thèse de Church-Turing

Proposition (Thèse de Church-Turing)

Un algorithme est automatisable si et seulement si il existe une machine de Turing qui permet de mettre en œuvre concrètement cet algorithme.

- **Une machine de Turing**

- ▶ Donne une description formelle et précise de l'entité *automate*
- ▶ Fournit un modèle abstrait du fonctionnement des appareils mécaniques de calcul, tel un ordinateur.

(Voir https://fr.wikipedia.org/wiki/Machine_de_Turing)

Langages formels

Chaînes de lettres, Langages formels

Définition (Chaînes de lettres)

Soit Σ un ensemble fini, appelé *alphabet*. L'ensemble Σ^* des chaînes (ou mots) de symboles de Σ est l'ensemble des suites finies d'éléments de Σ .

$$\Sigma^* = \{ \langle s_1, \dots, s_n \rangle \mid n \geq 0 \wedge \forall i : 1 \leq i \leq n : s_i \in \Sigma \}. \quad (1)$$

Remarque

Nous notons simplement les chaînes par juxtaposition des éléments de l'alphabet. Nous écrivons donc $s_1 \dots s_n$ au lieu de $\langle s_1, \dots, s_n \rangle$.

Définition (Langages formels)

Un **langage formel**² L sur un alphabet Σ est un ensemble de mots sur Σ . C'est-à-dire que tout langage formel est une partie de Σ^*

$$L \subseteq \Sigma^*$$

²Nous ne considérons pour le moment que la “forme” des mots, sans tenir compte de leurs significations (sémantiques).

Opérations (ensemblistes) sur les langages

Définition

Soient E et F deux sous-ensembles de Σ^* . L'union (notée " $\cup$ ") et la concaténation (notée " $\cdot$ ") de E et F , l'itération de E (notée " $\star$ "), ainsi que le complémentaire de E (notée " $\bar{\cdot}$ ") sont des sous-ensembles de Σ^* définis respectivement par

$$\begin{aligned} E \cup F &= \{x \in \Sigma^* \mid x \in E \vee x \in F\} \\ E.F &= \{x y \in \Sigma^* \mid x \in E \wedge y \in F\} \\ E^\star &= \left\{ \prod_{i=1}^n x_i \in \Sigma^* \mid x_1, \dots, x_n \in E, n \geq 0 \right\} \\ \overline{E} &= \{x \in \Sigma^* \mid x \notin E\} \end{aligned}$$

Remarques

- Le produit $\prod_{i=1}^n x_i$ correspond à la concaténation de suites de Σ^* , obtenue en juxtaposant les symboles des suites x_i :

$$\begin{aligned}\prod_{i=1}^n x_i &= x_1 \dots x_n \quad x_i = s_{1,i} \dots s_{m_i,i} \quad n \geq 0 \\ &= \underbrace{s_{1,1} \dots s_{m_1,1}}_{x_1} \underbrace{s_{1,2} \dots s_{m_2,2}}_{x_2} \dots \underbrace{s_{1,n} \dots s_{m_n,n}}_{x_n}\end{aligned}$$

- Le symbole $\star$ utilisé pour la fermeture itérative de E , est appelé étoile de Kleene et il faut veiller à ne pas le confondre avec le symbole $*$. En effet $E^\star$, est un ensemble de suites de symboles de Σ , alors que E^* est un ensemble de suites de suites de symboles de Σ .
- La relation d'inclusion des ensembles est une relation d'ordre partiel. Elle vérifie la propriété (appelée "*ordre naturel des éléments de* $\wp(\Sigma^*)$ "):

$$\forall E, F \in \wp(\Sigma^*) : E \subseteq F \iff E \cup F = F$$

Propriétés structurelles de $\wp(\Sigma^*)$

Soit Σ un ensemble fini.

- $(\wp(\Sigma^*), \cup, \cdot, \{\}, \{\lambda\})$ est un semi-anneau idempotent, i.e. :

1. $(\wp(\Sigma^*), \cup, \{\})$ est un monoïde idempotent commutatif.

$\forall E, F, G \in \wp(\Sigma^*) :$

$$E \cup (F \cup G) = (E \cup F) \cup G \quad (2a)$$

$$E \cup F = F \cup E \quad (2b)$$

$$E \cup \{\} = E \quad (2c)$$

$$E \cup E = E \quad (2d)$$

2. $(\wp(\Sigma^*), \cdot, \{\lambda\})$ est un monoïde. $\forall E, F, G \in \wp(\Sigma^*)$

$$E \cdot (F \cdot G) = (E \cdot F) \cdot G \quad (3a)$$

$$E \cdot \{\lambda\} = \{\lambda\} \cdot E = E \quad (3b)$$

3. $\{\}$ est absorbant pour l'opération $\cdot$. $\forall E \in \wp(\Sigma^*)$

$$E \cdot \{\} = \{\} \cdot E = \{\} \quad (4a)$$

4. L'opération $\cdot$ est distributive par rapport à $\cup$. $\forall E, F, G \in \wp(\Sigma^*) :$

$$E \cdot (F \cup G) = E \cdot F \cup E \cdot G \quad (5a)$$

$$(E \cup F) \cdot G = E \cdot G \cup F \cdot G \quad (5b)$$

Propriétés structurelles de $\wp(\Sigma^*)$ (suite 2/2)

Soit Σ un ensemble fini.

- La relation d'ordre partiel naturel $\subseteq$ sur $\wp(\Sigma^*)$ est monotone et compatible pour ces opérateurs. $\forall E, F, G \in \wp(\Sigma^*)$:

$$E \subseteq F \Rightarrow E \cup G \subseteq F \cup G \quad (6a)$$

$$E \subseteq F \Rightarrow E.G \subseteq F.G \quad (6b)$$

$$E \subseteq F \Rightarrow G.E \subseteq G.F \quad (6c)$$

$$E \subseteq F \Rightarrow E^* \subseteq F^* \quad (6d)$$

Preuve.

Propriétés des ensembles à montrer en exercices.



Langages réguliers

Expressions régulières

Définition (Expressions régulières)

L'ensemble **RegExpr** $_{\Sigma}$ des expressions régulières sur Σ , est formé de :

- Des expressions indécomposables, que sont : ϕ et λ
- Des expressions composées, que sont :
Etant donnée deux expressions régulières $expr_1$ et $expr_2$:
 - ▶ Union des expressions $expr_1$, $expr_2$, notée $expr_1 + expr_2$
 - ▶ Concaténation des expressions $expr_1$, $expr_2$, noté $expr_1 . expr_2$
 - ▶ Itération de l'expression $expr_1$ notée $expr_1^*$.

Exemple : Sur $\Sigma = \{a, b, c\}$, on a les expressions $a, b, a + b, a^*, \dots$

Question : que représente une expression régulière ?

Réponse : un langage régulier.

Sémantique des expressions régulières

Les expressions régulières définissent des ensembles de chaînes finies de lettres.

Définition (Fonction sémantique $\mathcal{L}$ des expressions régulières)

La fonction sémantique est définie inductivement, sur la structure des expressions régulières, par la fonction $\mathcal{L}$ (voir FIGURE 2).

$$\begin{array}{lcl} \mathcal{L} & : & \mathbf{RegExpr}_{\Sigma} \longrightarrow \wp(\Sigma^*) \\ \\ \mathcal{L} \llbracket \lambda \rrbracket & = & \{ \lambda \} \\ \mathcal{L} \llbracket \phi \rrbracket & = & \{ \} \\ \mathcal{L} \llbracket x \rrbracket & = & \{ x \} \\ \mathcal{L} \llbracket expr_1 + expr_2 \rrbracket & = & \mathcal{L} \llbracket expr_1 \rrbracket \cup \mathcal{L} \llbracket expr_2 \rrbracket \\ \mathcal{L} \llbracket expr_1 expr_2 \rrbracket & = & (\mathcal{L} \llbracket expr_1 \rrbracket) (\mathcal{L} \llbracket expr_2 \rrbracket) \\ \mathcal{L} \llbracket expr^* \rrbracket & = & (\mathcal{L} \llbracket expr \rrbracket)^* \end{array}$$

Figure: Fonction sémantique $\mathcal{L}$ des expressions régulières

Langages réguliers et expressions régulières

Définition (Langages réguliers)

Une partie $E \subseteq \wp(\Sigma^*)$ est un langage régulier sur Σ s'il existe une expression régulière $expr$ telle E soit l'image par $\mathcal{L}$ de cette expression:

$$E \text{ est un langage régulier} \Leftrightarrow \exists expr \in \mathbf{RegExpr}_\Sigma \mid E = \mathcal{L} \llbracket expr \rrbracket.$$

L'ensemble des langages réguliers $\mathbf{Reg}_\Sigma$ est, par définition, l'ensemble formé des images par $\mathcal{L}$ des expressions de $\mathbf{RegExpr}_\Sigma$. C'est donc le codomaine de $\mathcal{L}$:

$$\mathbf{Reg}_\Sigma \stackrel{def}{=} \{\mathcal{L} \llbracket expr \rrbracket \mid expr \in \mathbf{RegExpr}_\Sigma\} = \text{codom}(\mathcal{L}) \quad (7)$$

Propriété

L'ensemble des langages réguliers $\mathbf{Reg}_\Sigma$ est le plus petit ensemble de langages qui vérifie que :

1. $\phi, \{\lambda\} \in \mathbf{Reg}_\Sigma$
2. $\{a\} \in \mathbf{Reg}_\Sigma, \forall a \in \Sigma$
3. Si $A, B \in \mathbf{Reg}_\Sigma$ alors
 - 3.1 $A \cup B \in \mathbf{Reg}_\Sigma$
 - 3.2 $A \cap B \in \mathbf{Reg}_\Sigma$
 - 3.3 $A^* \in \mathbf{Reg}_\Sigma$

Langages rékursifs

Langages rékursivement énumérables

Ensembles dénombrables ou énumérables

Définition (Ensemble est dénombrable ou énumérable)

Un ensemble est dénombrable ou énumérable, s'il existe une bijection de cet ensemble vers une partie de l'ensemble des nombres entiers.

Exemple :

- Tout ensemble fini est dénombrable
- L'ensemble des mots sur un alphabet Σ fini est dénombrable
- L'ensemble des expressions régulières est dénombrable.

Langages rékursifs, langages récursivement énumérables

Définition (Langages récursivement énumérables)

Un langage formel L est *récursivement énumérable*, s'il existe un algorithme permettant d'énumérer les mots de ce langage.

Définition (Langages rékursifs)

Un langage formel L est *rékursif* s'il existe un algorithme qui permet de décider si un mot quelconque m de Σ^* est un mot de L . Le programme donne la réponse “oui” si $m \in L$, sinon il donne “non”.

Un algorithme qui permet de décider si un mot appartient à un langage ou pas est appelé un **algorithme de décision**.

Propriétés

Soit Σ un alphabet

- Tout langage L récursif est récursivement énumérable.
 - ▶ Il suffit de faire un brute force sur Σ^* et de donner chaque mot généré à l'algorithme de décision de L .
- Il existe des langages récursivement énumérables qui ne sont pas récursifs.
 - ▶ L'ensemble des tautologies en logiques formelles n'est pas récursif
- Il existe des langages formels qui ne sont pas récursivement énumérables.
 - ▶ Preuve : Théorème d'incomplétude de Gödel
 - ▶ **Conséquences**
 - ▶ Il existe des langages pour lesquels il n'est pas possible de déterminer ou non si certains mots sont dans ces langages.
 - ▶ Il y a des problèmes pour lesquelles on ne peut pas trouver d'algorithme automatisable.

Plan : 2 - Grammaires formelles de Chomsky

- 2.1 Grammaires formelles de Chomsky
- 2.2 Langage engendré par une grammaire
- 2.3 Classification de Chomsky

A quoi ça sert ?

Problème :

Etant donné un langage récursivement énumérable, comment générer précisément et automatiquement tous les mots de ce langage ?

Solution possible : on utilise une approche générative en appliquant systématiquement des règles (conventions) d'une grammaire, en partant d'un symbole (appelé pour cela symbole initial), et en s'arrêtant aux chaînes uniquement formées de symboles de l'alphabet (appelés pour cela symboles terminaux de ce processus de "dérivation").

Grammaire formelle de Chomsky

Définition (Grammaire formelle de Chomsky)

Une grammaire formelle de Chomsky est la donnée d'un quadruplet

$$\mathcal{G} = \langle \mathcal{N}, \Sigma, \mathcal{P}, S \rangle$$

- $\mathcal{N}$: ensemble des symboles appelés *non terminaux*,
- Σ : ensemble des symboles appelés *terminaux*.
- $\mathcal{P}$: ensemble de couples (règles de production de la grammaire) $\langle \alpha, \beta \rangle$ qu'on note sous la forme $\alpha \rightarrow \beta$, avec $\alpha, \beta \in (\mathcal{N} \cup \Sigma)^*$.
- S : symbole initial (qui est un non terminal).

La grammaire $\mathcal{G}$ définit un langage formel $\mathcal{L}$. Les mots de $\mathcal{L}$ sont formés des symboles terminaux de Σ , qui est donc l'alphabet de $\mathcal{L}$.

Conventions de notations

Les symboles terminaux :

sont notés à partir de lettres minuscules $a, b, c, \dots$

Les symboles non terminaux :

sont notés à partir de lettres majuscules $A, B, C, \dots$
ou encore par des chiffres $0, 1, \dots$

Le symbole initial : est noté à partir de la lettre majuscule S

Les chaînes formées de symboles terminaux ou non terminaux :

sont notées à partir des lettres grecques $\alpha, \beta, \dots$
(rappel : ces chaînes sont des éléments de $(\mathcal{N} \cup \Sigma)^*$)

La chaîne vide de symboles : est notée par le symbole grecque λ

La longueur de la chaîne α : est notée par $|\alpha|$,
c'est le nombre d'occurrences de symboles dans la chaîne

Pour simplifier les notations :

- On identifie tout symbole à la chaîne formée de cet unique symbole
- On note la concaténation de chaîne par simple juxtaposition de ces chaînes, en respectant l'ordre de la concaténation

Quels mots peut-on générer avec cette grammaire ?

La grammaire $\mathcal{G} = \langle \mathcal{N}, \Sigma, \mathcal{P}, S \rangle$ avec :

- $\mathcal{N} = \{S\}$
- $\Sigma = \{a, m\}$
- $\mathcal{P} = \{S \rightarrow maS, S \rightarrow \lambda\}$

Pour générer les mots du langage qui correspond à cette grammaire, nous partons du symbole initial S et nous appliquons systématiquement et méthodiquement toutes les règles de production de la grammaire.

L'application d'une règle de production transforme une chaîne de caractères en une autre chaîne de caractères, toutes deux formées de symboles terminaux ou non terminaux. On dit qu'on a “**dérivé**” une nouvelle chaîne de caractères.

Lorsqu'une suite de dérivations donne une chaîne uniquement formée de symboles terminaux, cette suite de dérivations s'arrête et on obtient alors un mot généré par cette grammaire.

Arbre de dérivation d'une grammaire

L'arbre de dérivation d'une grammaire $\mathcal{G}$ permet d'organiser les dérivations engendrant les mots du langage de la grammaire $\mathcal{G}$.

- Le racine de l'arbre contient le symbole initiale S . Les nœuds de l'arbre sont les chaînes de caractères obtenues en un nombre quelconque de dérivations de S .
- Les successeurs directs d'un nœud sont obtenus à partir de toutes les dérivations possibles en une étape de ce nœud.
- Les feuilles de l'arbre sont les chaînes qui ne sont plus dérivables. Les chaînes terminales de l'arbre sont les mots du langage généré par la grammaire $\mathcal{G}$.

Dérivation en une étape

Une dérivation en une étape, notée $\xRightarrow[\mathcal{G}]{}$, est une relation binaire sur $(\mathcal{N} \cup \Sigma)^*$. Elle transforme une chaîne de caractères en une autre chaîne de caractères, toutes deux formées de symboles terminaux ou non terminaux.

Définition (Dérivation en une étape)

Soient $\alpha, \beta \in (\mathcal{N} \cup \Sigma)^*$.

β est dérivable en une étape à partir de α , noté $\alpha \Longrightarrow \beta$, si et seulement si il existe une règle de production $\alpha' \longrightarrow \beta'$ telle que α' et β' sont respectivement des sous-chaînes de α et β , et on obtient la chaîne β en remplaçant α' par β' dans α .

Formellement :

$$\alpha \Longrightarrow \beta \quad \Leftrightarrow \quad \exists \pi, \sigma, \alpha', \beta' \in (\mathcal{N} \cup \Sigma)^* : \begin{cases} \alpha &= \pi \alpha' \sigma \\ \beta &= \pi \beta' \sigma \\ \alpha' &\longrightarrow \beta' \in \mathcal{P} \end{cases}$$

Séquences de dérivations successives

On s'intéresse cette fois-ci à la formalisation des séquences de dérivation successives. Nous définissons tout d'abord une dérivation en un nombre n d'étapes d'une chaîne, pour ensuite définir une dérivation en un nombre quelconque d'étapes (ou plus simplement la notion de "dérivation").

Définition (Dérivation en n étapes)

Soient $\alpha, \beta \in (\mathcal{N} \cup \Sigma)^*$ et $n \in \mathbb{N}$, β est dérivable en n étapes à partir de α , noté $\alpha \xRightarrow{n} \beta$, s'il existe $\alpha_0, \alpha_1, \dots, \alpha_i, \dots, \alpha_n \in (\mathcal{N} \cup \Sigma)^*$ telles que

$$\alpha_0 = \alpha \quad \text{et} \quad \alpha_0 \Longrightarrow \alpha_1 \Longrightarrow \dots \Longrightarrow \alpha_i \Longrightarrow \dots \Longrightarrow \alpha_n \quad \text{et} \quad \alpha_n = \beta$$

Définition (Dérivation en un nombre quelconque étapes)

Soient $\alpha, \beta \in (\mathcal{N} \cup \Sigma)^*$, β est dérivable en un nombre quelconque d'étapes (ou dérivable) à partir de α , noté $\alpha \xRightarrow{*} \beta$, s'il existe $n \in \mathbb{N}$ tel que $\alpha \xRightarrow{n} \beta$.

Séquences de dérivations successives

On s'intéresse cette fois-ci à la formalisation des séquences de dérivation successives. Nous définissons tout d'abord une dérivation en un nombre n d'étapes d'une chaîne, pour ensuite définir une dérivation en un nombre quelconque d'étapes (ou plus simplement la notion de "dérivation").

Définition (Dérivation en n étapes)

Soient $\alpha, \beta \in (\mathcal{N} \cup \Sigma)^*$ et $n \in \mathbb{N}$, β est dérivable en n étapes à partir de α , noté $\alpha \xRightarrow{n} \beta$, s'il existe $\alpha_0, \alpha_1, \dots, \alpha_i, \dots, \alpha_n \in (\mathcal{N} \cup \Sigma)^*$ telles que

$$\alpha_0 = \alpha \quad \text{et} \quad \alpha_0 \Longrightarrow \alpha_1 \Longrightarrow \dots \Longrightarrow \alpha_i \Longrightarrow \dots \Longrightarrow \alpha_n \quad \text{et} \quad \alpha_n = \beta$$

Définition (Dérivation en un nombre quelconque étapes)

Soient $\alpha, \beta \in (\mathcal{N} \cup \Sigma)^*$, β est dérivable en un nombre quelconque d'étapes (ou dérivable) à partir de α , noté $\alpha \xRightarrow{*} \beta$, s'il existe $n \in \mathbb{N}$ tel que $\alpha \xRightarrow{n} \beta$.

Langage engendré par une grammaire

Définition (Langage engendré par une grammaire)

Le langage engendré par la grammaire $\mathcal{G}$, est l'ensemble de toutes les chaînes terminales qu'on obtient en effectuant toutes les dérivations en des nombres quelconques d'étapes du symbole initial S .

$$\mathcal{L}[\mathcal{G}] = \left\{ \alpha \in \Sigma^* \mid S \xRightarrow{*} \alpha \right\}$$

Classification de Chomsky

Les grammaires de Chomsky servent à définir les langages formels récursivement énumérables.

On a quatre classes de grammaires de plus en plus restrictives (On obtient ça en fixant des restrictions sur les règles)

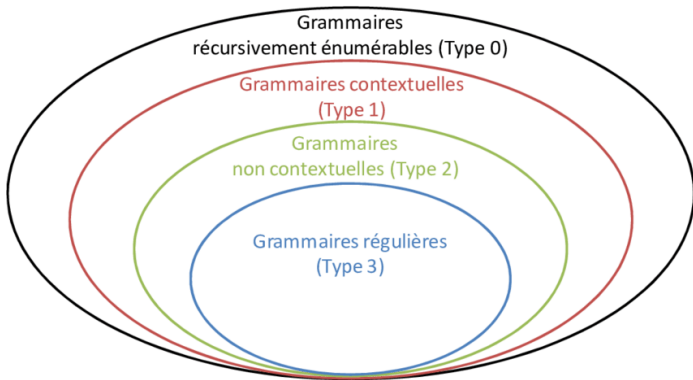


Figure: Classification des grammaires par Chomsky

Type 0 : grammaires générales

C'est l'ensemble de toutes les grammaires possibles et imaginables

- Elles permettent de générer tous les langages récursivement énumérables
- Les règles de productions s'écrivent sous la forme $\alpha \longrightarrow \beta$ telles que α contient au moins un non terminal
- Le problème de construire un analyseur syntaxique pour une de ces grammaires est indécidable

Type 1 : grammaires Context-Dependent (CD) (1/2)

Définition (Grammaire CD / définition 1)

Une grammaire $\mathcal{G}$ est une grammaire CD si et seulement si toutes ses règles de productions $\alpha \longrightarrow \beta$ sont telles que $|\alpha| \leq |\beta|$.

Définition (Grammaire CD / définition 2)

Une grammaire $\mathcal{G}$ est CD si et seulement si toutes ses règles de productions $\alpha \longrightarrow \beta$ sont telles que $\alpha = \pi X \sigma, \beta = \pi \gamma \sigma$ avec $|\gamma| \geq 1$.

Remarque

Ces grammaires sont dites Context-Dependant parce que le contexte d'une production $\langle \pi, \sigma \rangle$ est conservé dans la substitution.

Exercice

Montrer que la définition 1 est équivalente à la définition 2.

Type 1 : grammaires Context-Dependent (CD) (2/2)

Propriété

Le langage d'une grammaire CD est récursif.

Preuve.

On génère toutes les chaînes de longueurs inférieures à la longueur du mot courant et les compare au mot courant.



Type 2 : grammaires Context-Free (CF)

Définition (Grammaire CF)

Une grammaire $\mathcal{G}$ est CF si et seulement si toutes ses règles de productions sont de la forme $X \rightarrow \alpha, \alpha \in (\mathcal{N} \cup \Sigma)^*$

Remarque

- Contrairement aux grammaires CD, on peut avoir $\alpha = \lambda$ si $|\alpha| = 0$.
- Ces grammaires sont dites Context-Free parce qu'on peut remplacer un non-terminal sans tenir compte du contexte de celui-ci.
- Ces grammaires permettent de définir des analyseurs syntaxiques.

Type 3 : grammaires linéaires à droite

Définition (Grammaire linéaire à droite)

Une grammaire $\mathcal{G}$ est régulière si et seulement si toutes ses règles de productions sont de la forme $X \longrightarrow \alpha$ ou $X \longrightarrow \alpha Y, \alpha \in (\mathcal{N} \cup \Sigma)^*$

Ces grammaires permettent de définir les langages réguliers.